

## Práctica Técnica Calificada 03 (PC03)

### A. Datos Generales

- **Curso:** Sistemas Distribuidos
- **Evaluación:** Práctica Técnica Calificada 03 (PC03)
- **Unidad Evaluada:** Unidad 3 - Sincronización y tolerancia de fallas
- **Sesiones comprendidas:** Sesión 23 a Sesión 29
- **Duración de la evaluación:** 90 minutos
- **Caso de Estudio:** El curso esta fácil – fallo en cascada (Basado en `mqtt-simulador`)

### B. Objetivo de la evaluación:

Evaluar la capacidad del estudiante para analizar, diseñar y justificar arquitecturas distribuidas aplicadas a contextos reales, basándose en criterios técnicos y funcionales.

### C. Instrucciones para el Estudiante

- Lee atentamente el caso propuesto.
- Implemente cada decisión técnica.
- Sé claro, preciso y coherente.

### D. RAZONAMIENTO EN AMBIENTES DISTRIBUIDOS

Revisa y analiza el código del repositorio, proporcionado con mucha antelación.

### Fase 0: Despliegue

**Objetivo:** Escalar la complejidad. Con 2 nodos es fácil ponerse de acuerdo (Ing. Este caso esta como la tabla del 1); enronces le vamos a subir el "ruido", escalaremos a 5 nodos.

#### Instrucciones:

Su sistema actual incluirá mejoras. Modifiquen el archivo `docker-compose.yml` para desplegar **5 Publicadores (Sensores)** en total. Cada uno con una prioridad única (10, 20, 30, 40, 50) y un ID único. El Coordinador original (`Lock-Coordinator`) temporalmente queda desactivado; ahora CUALQUIERA de los 5 sensores debe ser capaz de convertirse en líder.

Acciones rápidas, no te detangas esto forma parte del reto:

Debe copiar/pegar el servicio publisher 5 veces cambiando variables:

- `publisher-1: PRIORITY=10, ID=sensor-1`
- ...
- `publisher-5: PRIORITY=50, ID=sensor-5`
- **Reto:** Todos deben tener la variable `ELECTION_PARTICIPANTS=sensor-1:10,sensor-2:20,sensor-3:30,sensor-4:40,sensor-5:50`.

### Fase 1: Sincronización.

**El Problema:** En una red de 5 nodos, la probabilidad de que uno tenga el reloj mal configurado es del 100%. Si un nodo envía datos con `timestamp` incorrecto, corrompe el orden causal del sistema.

#### Instrucciones:

1. **Implementar Cristian Mejorado:** En `publisher.js`, el sensor no solo debe pedir la hora, sino calcular el RTT (Round Trip Time). Si `RTT > 500ms`, la sincronización se considera "inadecuada" y se descarta.
2. **Barrera de Entrada Temporal:** En `persistence-subscriber.js`, implementar un filtro estricto.

- Si  $|HoraMensaje - HoraLocal| > 2s \rightarrow$  **DESCARTAR**.
- Excepción: Si el **Reloj Lógico (Lamport)** del mensaje es Mayor que el último visto Y el vector clock es consistente, se acepta pero se fuerza la HoraLocal.

**Condiciones:** Se suministrará un script de Chaos (ver líneas abajo), que inyectará un nodo con fecha del año 2030. El sistema no debe caerse ni guardar ese dato.

## Fase 2: El Consenso.

**Requisito:** Haber superado la Fase 1 (relojes estables).

**El Problema:** Split-Brain.

Con 5 nodos, si el Líder (Prioridad 50) tiene un "lag" de red, el nodo 40 intentará ser líder. El 50 vuelve y ambos creen mandar.

### Instrucciones:

1. **Implementar "Leases" (Arrendamientos):** El liderazgo no es eterno. Dura 5 segundos. El líder debe renovarlo escribiendo en un tópico retenido /election/lease cada 2 segundos.
2. **Regla del Asesino Silencioso:** Si un nodo (ej. 40) no ve una renovación del lease en 5s, inicia elección.
3. **El Quórum:** Para declararse líder, el ganador debe recibir ACK de al menos **3 de los 5 nodos** (Mayoría simple  $N/2 + 1$ ). Sin mayoría, el nodo entra en modo CANDIDATE pero no LEADER, y no otorga recursos.

**Condiciones:** Se aislará al líder de la red. Los 4 restantes deben elegir uno nuevo. Cuando el viejo líder vuelva, debe darse cuenta de que su lease expiró y convertirse en seguidor inmediatamente.

## Fase 3: Resilencia.

**Requisito:** Haber superado la Fase 2 (un líder único y legítimo).

**El Problema:** "Amnesia Total".

El contenedor del líder explota (Crash). Docker lo reinicia. El nuevo líder (o el mismo) aparece con la memoria RAM vacía. La cola de procesos esperando el recurso se pierde.

### Instrucciones:

1. **Write-Ahead-Log (WAL):** En el código del líder (dentro de publisher.js), cada vez que alguien entra a la waitingQueue, se debe hacer un `fs.appendFileSync('wal.log', ...)` con la operación.
2. **Recuperación al Inicio:** Al ganar una elección (Fase 2), el nuevo líder debe leer wal.log (si existe en un volumen compartido o simularlo localmente reconstruyendo estado) ANTES de abrir el servicio de Mutex.

**Condiciones:** Se mata al líder con 3 sensores en cola. Al revivir (o elegirse otro), debe imprimir: [RECOVERY] Restored queue: [sensor-2, sensor-3].

## Fase de validación.

### El Script del Caos: chaos-ultimate.sh

Con este script, Ud. Hará las pruebas necesarias y podrá correrlo cuantas veces deseen, pero solo cuenta la ejecución final.

Crea un archivo chaos-ultimate.sh en la raíz del proyecto:

```
#!/bin/bash

# Colores
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

echo -e "${YELLOW} INICIANDO PROYECTO EL CURSO FACILITO: PROTOCOLO DE EVALUACIÓN
${NC}"
echo "Asegúrese de tener 5 publishers corriendo: publisher-1 al publisher-5"
sleep 2

# =====
# NIVEL 1: RESGUARDANDO EL TIEMPO
# =====
echo -e "\n${YELLOW}[FASE 1] Probando Integridad Temporal...${NC}"

# 1. Inyectamos un mensaje malicioso (Time Travel Attack)
echo "    -> Inyectando mensaje del año 2050..."
docker exec mqtt-broker mosquitto_pub -t "sensors/telemetry" -m '{"deviceId":
"malicious-node", "timestamp": "2050-01-01T00:00:00Z", "temperatura": 1000}'

sleep 2
# Verificamos si el suscriptor lo rechazó
if docker logs persistence-subscriber 2>&1 | grep -q "Rejected future packet";
then
    echo -e "${GREEN} ÉXITO: El ataque temporal fue neutralizado.${NC}"
else
    echo -e "${RED} FALLO: El sistema aceptó datos corruptos o no logueó el
rechazo.${NC}"
    exit 1
fi

# =====
# NIVEL 2: SPLIT-BRAIN
# =====
echo -e "\n${YELLOW}[FASE 2] Probando Estabilidad de Liderazgo (Quórum)...${NC}"

# Identificar al líder actual (asumimos que el log imprime "I am LEADER")
LEADER_CONTAINER=$(docker ps | grep publisher | head -n 1 | awk '{print $NF}')
echo "    -> Líder actual detectado (aprox): $LEADER_CONTAINER"

# Pausamos al líder (simulamos partición de red/congelamiento, no muerte aun)
echo "    -> Congelando al líder (SIGSTOP)..."
docker pause $LEADER_CONTAINER
sleep 7 # Esperamos más que el Lease (5s)

# Verificamos si alguien más tomó el mando
NEW_LEADER_LOGS=$(docker logs publisher-1 2>&1 | grep "Ascendido a Lider" | tail
-n 1)

if [ -z "$NEW_LEADER_LOGS" ]; then
    # Intentamos ver otros logs
    echo "    -> Buscando nuevo líder..."
fi
```

```
# Descongelamos
docker unpause $LEADER_CONTAINER
sleep 2

# Verificamos que el viejo líder NO siga creyéndose líder (debe haber visto que
perdió el lease)
if docker logs $LEADER_CONTAINER --tail 10 | grep -q "Stepping down"; then
    echo -e "${GREEN}      ÉXITO: Transición de poder ordenada y recuperación de
Split-Brain.${NC}"
else
    echo -e "${YELLOW} ALERTA: Verificar manualmente si hay dos líderes. Buscando
'Stepping down' en logs.${NC}"
fi

# =====
# NIVEL 3: PERSISTENCIA
# =====
echo -e "\n${YELLOW}[FASE 3] Probando Recuperación de Estado (WAL)...${NC}"

# Forzamos una cola (simulado enviando peticiones rapidas)
echo "    -> Generando tráfico para llenar la cola..."
docker exec publisher-2 node -e "require('./config').topics..." > /dev/null 2>&1
&

# Matamos violentamente al contenedor que sea líder
echo "    -> KILL -9 al Líder..."
# (Aquí simplificamos matando al publisher-5 que suele ser el de mayor prioridad)
docker restart publisher-5
sleep 5

# Verificamos si recuperó la memoria
if docker logs publisher-5 --tail 50 | grep -q -E "WAL|Restored|Recovered"; then
    echo -e "${GREEN}      ÉXITO: El sistema recordó el estado previo a la
muerte.${NC}"
else
    echo -e "${RED}      FALLO: El líder revivió con Amnesia (sin logs de
recuperación).${NC}"
    exit 1
fi

echo -e "\n${YELLOW} EVALUACIÓN COMPLETADA ${NC}"
```

## Anexo A: Protocolo de Entrega y Control de Versiones

Para simular un entorno de desarrollo, se prohíbe trabajar directamente sobre la rama main.

### Instrucciones

Antes de escribir una sola línea de código, deben aislar su entorno de trabajo.

Toda la implementación del Proyecto debe residir en una rama específica (Feature Branch). El 'deploy' final será un Pull Request.

Se exige **al menos un commit por Fase completada** con mensajes semánticos.

- feat(time): implementada validación de timestamp y cristian robusto
- feat(consensus): añadido lease y logica de quorum 3/5
- feat(persistence): integrado WAL para recuperación de crash

## Anexo B: Pruebas de Validez

El estudiante debe demostrar que entiende por qué pasó. Se les pedirá crear un archivo EVIDENCIAS.md en la raíz del proyecto con el siguiente contenido:

```
# EVIDENCIA DE VALIDACIÓN - PROYECTO

## FASE 1: Defensa Temporal.
**Prueba:** Inyección de dato anómalo, del año 2050.
**Resultado Esperado:** El Persistence Subscriber rechaza el dato.
**Log Capturado:**
> [Copiar aquí la línea del log del contenedor persistence-subscriber donde dice "WARN: Rejected future packet..."]

## FASE 2: Consenso y Split-Brain
**Prueba:** Pausa del Líder actual.
**Resultado Esperado:** Elección de nuevo líder por mayoría (3/5).
**Log Capturado:**
> [Copiar aquí log del Nuevo Líder: "Won election with 3 votes"]
> [Copiar aquí log del Viejo Líder al volver: "Lease expired. Stepping down"]

## FASE 3: Persistencia WAL
**Prueba:** Reinicio forzado (Kill) del Coordinador con cola llena.
**Resultado Esperado:** Restauración de la cola al iniciar.
**Log Capturado:**
> [Coloca aquí log de inicio: "[WAL] Replaying 5 events... Queue restored."]
```

## Anexo C: Recomendaciones

1. **Logs en Tiempo Real:** No programen a ciegas. Mantengan siempre una terminal abierta con `docker logs -f [contenedor_lider]` para ver lo que está pasando en vivo.
2. **Divide y Vencerás:** No intenten hacer el WAL (Fase 3) si la Elección (Fase 2) aún falla. Un sistema persistente que elige dos líderes es un desastre persistente.
3. **Cuidado con el JSON:** El 90% de los fallos en la Fase 1 son por intentar leer propiedades de un `payload` mal parseado. Usen `try-catch` siempre al procesar mensajes MQTT.
4. **La Red no es Instantánea:** Al probar los *leases* y *heartbeats*, denle margen al sistema. No pongan `timeouts` de 10ms; Docker en sus laptops, hagan uso 2s o 5s.